

1. Wstęp i opis problemu badawczego

W ramach niniejszego projektu ponownie rozpatrujemy problem komiwojażera (ang. *Traveling Salesman Problem*, TSP), lecz tym razem zamiast metod przybliżonych, skupimy się na algorytmie podziału i ograniczeń (ang. Branch and Bound, B&B). TSP to klasyczny przykład problemu, w którym liczba możliwych tras rośnie w tempie silniowym, co sprawia, że zwykły przegląd wszystkich opcji bardzo szybko staje się niewykonalny [1]. Naszym głównym celem nie jest więc tylko znalezienie najkrótszej drogi, ale przede wszystkim sprawdzenie, jak metoda B&B radzi sobie z ograniczaniem tej ogromnej przestrzeni poszukiwań w praktyce.

Skupiamy się przede wszystkim na porównaniu trzech strategii przeszukiwania drzewa rozwiązań: BFS, DFS oraz Best-First. Chcemy sprawdzić, która z tych metod najskuteczniej „odcina” nieobiecujące gałęzie, chroniąc komputer przed wyczerpaniem pamięci i zbyt długim czasem obliczeń. Badania obejmują zarówno wariant symetryczny, jak i asymetryczny (ATSP), co pozwoli ocenić, czy kierunkowość kosztów ma realny wpływ na to, jak szybko algorytm jest w stanie wyznaczyć wynik optymalny [2][3].

1.1 Cel projektu i zakres badań

Głównym celem projektu jest opracowanie i implementacja algorytmu Branch-and-Bound (B&B) wspierającego trzy strategie przeszukiwania przestrzeni rozwiązań:

- Przeszukiwanie wszerz (BFS – Breadth-First Search).
- Przeszukiwanie w głąb (DFS – Depth-First Search).
- Przeszukiwanie według najniższego kosztu (Best-First Search).

Zakres badań obejmuje analizę złożoności obliczeniowej oraz wydajności algorytmu w zależności od wybranej metody przeglądu. Kluczowym elementem jest zbadanie wpływu górnego ograniczenia (Upper Bound) na redukcję liczby generowanych węzłów (złożoność pamięciową) oraz skrócenie czasu obliczeń.

1.2 Charakterystyka danych testowych i metodologia

Badania przeprowadzono przy użyciu dwóch rodzajów danych:

- **Instancje losowe:** Wygenerowano po 20 instancji dla każdego z co najmniej 7 rozmiarów problemu, z podziałem na warianty TSP i ATSP.
- **Baza TSPLIB:** Wykorzystano co najmniej 5 rzeczywistych instancji symetrycznych oraz 5 asymetrycznych w celu weryfikacji algorytmów na danych o różnej charakterystyce.

Wszystkie testy przeprowadzono na tych samych instancjach dla każdej metody, co zapewnia rzetelność porównania wyników. Przyjęto kryterium rozsądnego czasu, zdefiniowanego jako 15 minut na znalezienie rozwiązania optymalnego.

1.3 Hipotezy badawcze i złożoność

Projekt opiera się na założeniu, że dobór strategii przeszukiwania oraz jakość ograniczeń determinują praktyczną użyteczność metody B&B. Sformułowano następujące hipotezy:

1. Strategia Best-First Search wykaże **najwyższą efektywność w ograniczaniu liczby odwiedzonych węzłów**, co pozwoli na najszybsze dotarcie do wyniku optymalnego przy minimalnym przeszukiwaniu zbędnych gałęzi [4].
2. Zastosowanie **Upper Bound** drastycznie zredukuje przestrzeń poszukiwań, szczególnie w wariancie BFS, poprzez wczesne odcinanie gałęzi o koszcie przekraczającym znaleziony rekord.
3. Strategia BFS okaże się najmniej efektywna w kontekście zarządzania zasobami sprzętowymi. Zakłada się, że ze względu na brak priorytetyzacji węzłów, to właśnie ta metoda najszybciej osiągnie barierę 15 minut.

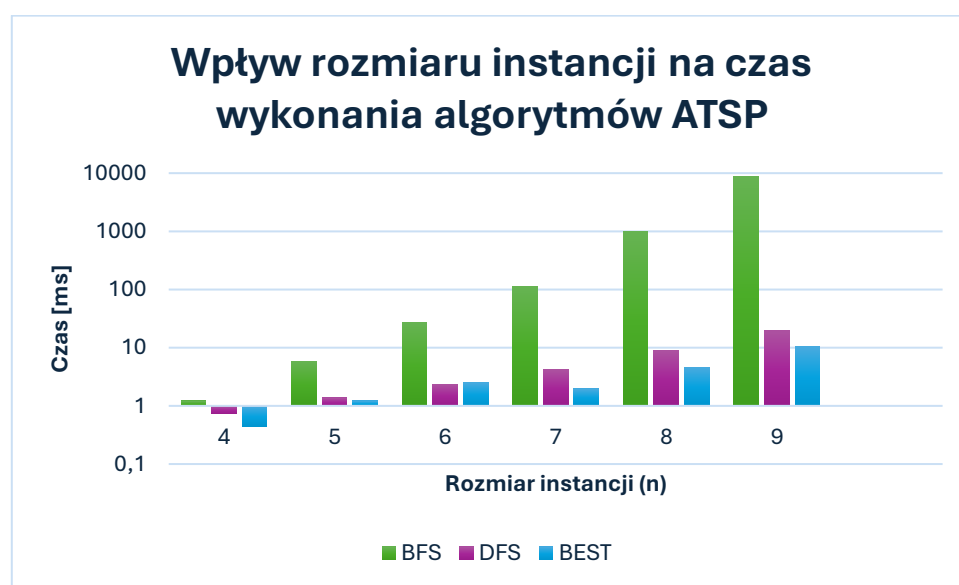
Efektywność metody podziału i ograniczeń w problemie TSP jest silnie uzależniona od ciasności oszacowań dolnych (Lower Bound), co w niniejszej pracy realizowane jest poprzez redukcję macierzy kosztów [5].

2. Badania i analiza wydajności algorytmów

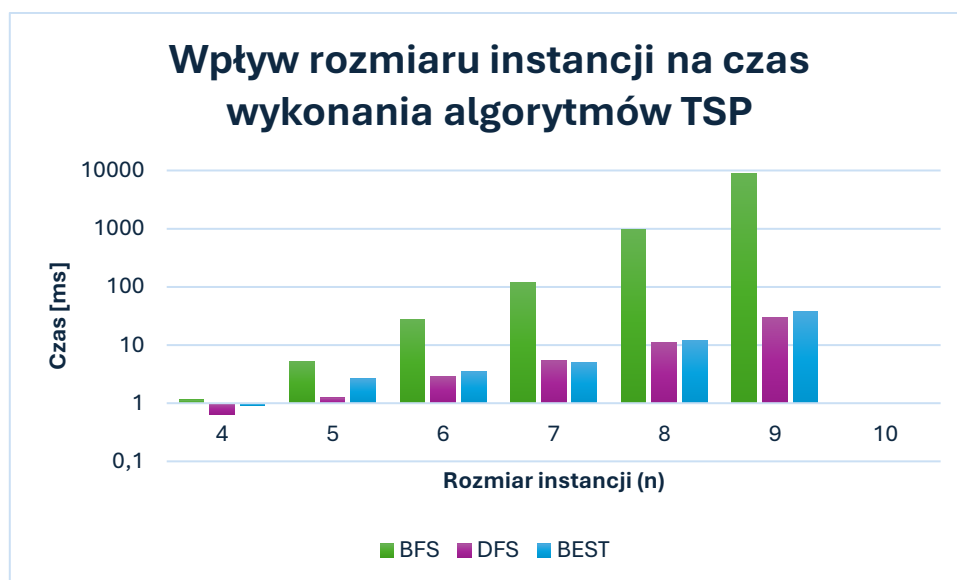
W tej części pracy przedstawiono wyniki testów wydajnościowych zaimplementowanych algorytmów, skupiając się na ich skalowalności oraz wpływie optymalizacji na czas obliczeń. Aby rzetelnie ocenić możliwości przygotowanego oprogramowania, wszystkie pomiary przeprowadzono na komputerze przenośnym wyposażonym w procesor Intel Core i5 generacji 8 oraz 16 GB pamięci RAM, pracującym pod kontrolą systemu Windows 10.

2.1 Ogólne porównanie algorytmów

W poniższym rozdziale dokonano zestawienia trzech wcześniej wspomnianych zaimplementowanych strategii przeglądania przestrzeni stanów. Pomiary przeprowadzono na instancjach losowych o rozmiarach od $n=4$ do $n=10$. Zastosowanie skali logarytmicznej na wykresach (Rys. 1 i Rys. 2) było niezbędne, ponieważ tylko w ten sposób można czytelnie przedstawić wyniki metod DFS i Best-First na tle ogromnych czasów generowanych przez BFS.



Rys 1. Wpływ rozmiaru instancji na czas wykonania algorytmów ATSP bez uwzględnienia granicy górnej.



Rys 2. Wpływ rozmiaru instancji na czas wykonania algorytmów TSP bez uwzględnienia granicy górnej.

Najważniejszą obserwacją wynikającą z przeprowadzonych badań jest drastyczna dysproporcja w wydajności algorytmu BFS w porównaniu do pozostałych metod. Podczas gdy BFS dla instancji o rozmiarze $n=10$ potrzebuje blisko 90 sekund na znalezienie rozwiązania, metody DFS i Best-First wykonują to samo zadanie w czasie poniżej 0,1 sekundy. Potwierdza to hipotezę o niskiej przydatności strategii wszerek w problemach o złożoności silniowej bez zastosowania bardzo silnych mechanizmów odcinania. Warto również zauważyć, że choć DFS i Best-First mają zbliżone wyniki na tak małych instancjach, to strategia Best-First wykazuje nieco większą stabilność, szczególnie w wariancie asymetrycznym (ATSP).

2.2 Porównanie instancji symetrycznych (TSP) i asymetrycznych (ATSP)

Na podstawie zebranych danych (Rys. 1 i Rys. 2) można stwierdzić, że instancje symetryczne (TSP) stanowią większe wyzwanie dla badanych algorytmów niż ich asymetryczne odpowiedniki. Różnica ta jest szczególnie widoczna w przypadku algorytmów DFS oraz Best-First dla największej badanej instancji ($n=10$):

Tabela 1. Porównanie średniego czasu wykonania [ms] dla instancji symetrycznych i asymetrycznych ($n=10$).

Typ instancji	BFS [ms]	DFS [ms]	Best-First [ms]
ATSP (Asymetryczny)	$8,93 \cdot 10^5$	$4,18 \cdot 10^1$	$1,92 \cdot 10^1$
TSP (Symetryczny)	$8,9 \cdot 10^5$	$8,6 \cdot 10^1$	$9,39 \cdot 10^1$

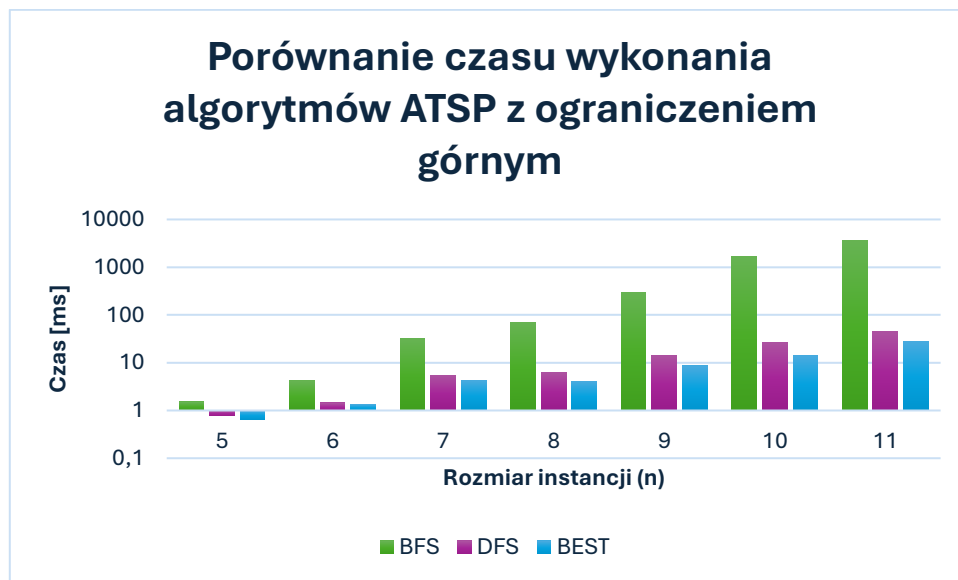
Źródło: opracowanie własne

Wyniki jednoznacznie pokazują, że wariant symetryczny (TSP) stanowi dla badanych algorytmów znacznie większe wyzwanie niż wersja asymetryczna. Wynika to głównie z symetrii kosztów, która utrudnia generowanie „ciasnych” ograniczeń i w konsekwencji zmusza program do przeszukiwania znacznie większej części drzewa stanów. W takiej sytuacji w wariancie TSP to często DFS osiąga lepszą wydajność, ponieważ unika narzutu na obsługę rozbudowanej kolejki priorytetowej w środowisku wielu identycznych kosztów. Z kolei w wariancie asymetrycznym (ATSP) bezsprzecznym liderem pozostaje Best-First, który dzięki dużemu zróżnicowaniu wag krawędzi potrafi precyzyjnie wybierać najbardziej obiecujące ścieżki i szybciej „docinać” drzewo. Należy jednak zauważyć, że niezależnie od tego, czy mamy do czynienia z instancją symetryczną czy asymetryczną, stała i ekstremalnie wysoka złożoność

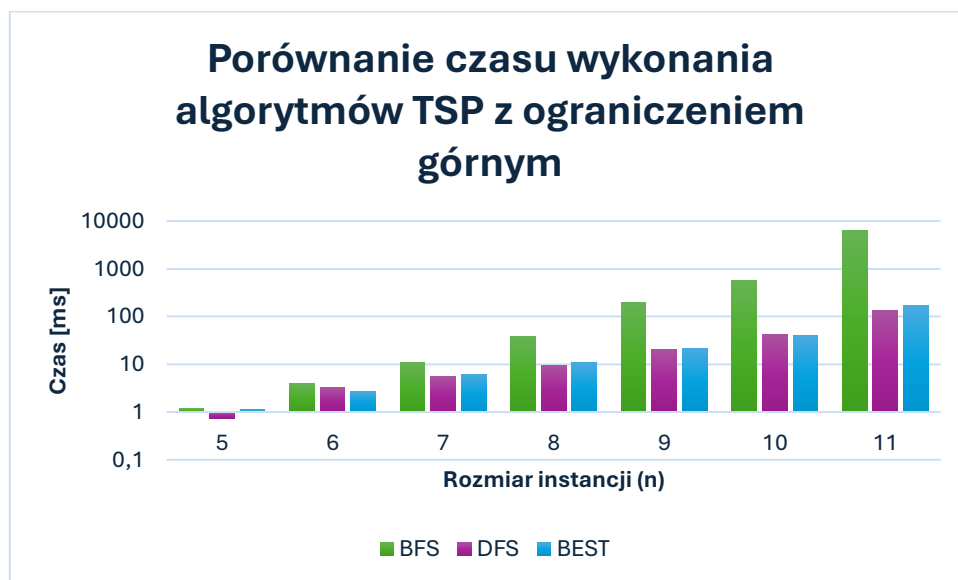
metody BFS dowodzi, iż struktura macierzy nie ma realnego wpływu na poprawę efektywności tej strategii.

2.3 Wpływ górnego ograniczenia na algorytmy

Kolejnym kluczowym etapem badań było sprawdzenie, jak wprowadzenie ograniczenia górnego (Upper Bound), wyznaczonego metodą najbliższego sąsiada (NN), wpłynie na wydajność algorytmu B&B. Zastosowanie tej optymalizacji pozwoliło na realne rozszerzenie zakresu badań i przeprowadzenie testów dla instancji o rozmiarach od $n=5$ do $n=11$.



Rys 3. Wpływ rozmiaru instancji na czas wykonania algorytmów ATSP z uwzględnieniem granicy górnej.



Rys 4. Wpływ rozmiaru instancji na czas wykonania algorytmów ATSP bez uwzględnienia granicy górnej.

Kluczowym wnioskiem z zestawienia wyników bazowych (Rys 1,2) z zoptymalizowanymi (Rys 3,4) jest drastyczna redukcja złożoności obliczeniowej niemal wszystkich badanych algorytmów po zastosowaniu wstępnego ograniczenia górnego (UB), wyznaczonego metodą najbliższego sąsiada (NN).

Szczególną uwagę zwraca przypadek algorytmu BFS, który bez optymalizacji osiągał barierę 90 sekund dla instancji $n=10$, natomiast po wprowadzeniu UB czas ten został zredukowany do poziomu ok. 1,5 sekundy. W celu precyzyjnego zobrazowania skali uzyskanych optymalizacji, w Tabeli 1 zestawiono wyniki dla reprezentatywnej instancji o rozmiarze $n=10$.

Tabela 2. Porównanie średniego czasu wykonania [ms] oraz ilości węzłów dla instancji symetrycznych i asymetrycznych z i bez ograniczenia górnego ($n=10$).

Wariant instancji	BFS (t[ms] / węzły)	DFS (t[ms] / węzły)	BEST (t[ms] / węzły)
ATSP (bez UB)	$8,93 \cdot 10^4 / 362\ 880$	$4,1 \cdot 10^{16} / 37$	$1,92 \cdot 10^1 / 620$
ATSP (z UB)	$1,68 \cdot 10^3 / 2\ 677$	$2,69 \cdot 10^1 / 18$	$1,39 \cdot 10^1 / 81$
TSP (bez UB)	$8,99 \cdot 10^4 / 362\ 880$	$8,60 \cdot 10^1 / 37$	$9,39 \cdot 10^1 / 195$
TSP (z UB)	$5,72 \cdot 10^2 / 835$	$4,25 \cdot 10^1 / 17$	$3,99 \cdot 10^1 / 127$

Źródło: opracowanie własne

Prezentowane dane potwierdzają, że wprowadzenie górnego ograniczenia (UB) drastycznie redukuje liczbę generowanych węzłów w algorytmie BFS z wartości pełnej silni (362 880) do zaledwie kilkuset, co skutkuje blisko 98-procentowym spadkiem czasu obliczeń. W przypadku metod DFS i Best-First, zastosowanie wstępnego rekordu pozwoliło na ponad dwukrotną poprawę wydajności czasowej dla TSP przy jednoczesnym znacznym obniżeniu złożoności pamięciowej, mierzonej liczbą aktywnych węzłów dla obu przypadków (TSP, ATSP).

2.4 Weryfikacja algorytmu na instancjach z biblioteki TSPLIB

W ramach końcowego etapu badań podjęto próbę weryfikacji wszystkich zaimplementowanych metod (BFS, DFS, Best-First) na wzorcowych instancjach z bazy TSPLIB, takich jak br17, gr17, ftv33 czy p43. Intencją testów było potwierdzenie skalowalności algorytmu na rzeczywistych danych o zróżnicowanej strukturze.

Tabela 2: Czasy wykonania [ms] dla instancji TSPLIB.

	Typ	Rozmiar	UB_Mode	BFS	BFS Węzły	DFS	DFS Węzły	BEST	BEST Węzły	Koszt
tsp	gr17	17	Z_UB	$5,08 \cdot 10^5$	380866	$3,18 \cdot 10^4$	72	$5,89 \cdot 10^4$	460547	2085
	gr17	17	BEZ_UB	-	-	$5,17 \cdot 10^4$	126	$2,64 \cdot 10^5$	589564	2085
	gr21	21	Z_UB	-	-	$2,46 \cdot 10^4$	97	$2,91 \cdot 10^4$	390404	1474
	gr21	21	BEZ_UB	-	-	$4,78 \cdot 10^4$	191	$3,60 \cdot 10^4$	443181	1474
	gr24	24	Z_UB	-	-	$4,72 \cdot 10^4$	482	$3,15 \cdot 10^4$	215 630	1272
	fri26	26	-	-	-	-	-	-	-	-
atsp	bays29	29	-	-	-	-	-	-	-	-
	br17	17	-	-	-	-	-	-	-	-
	ftv33	33	-	-	-	-	-	-	-	-
	ftv35	35	-	-	-	-	-	-	-	-
	ftv38	38	-	-	-	-	-	-	-	-
	p43	43	-	-	-	-	-	-	-	-

Źródło: opracowanie własne

Obecność symboli „-” w tabeli wynika bezpośrednio z przekroczenia narzuconego limitu 15 minut lub wyczerpania pamięci RAM przy próbie przetworzenia ogromnej liczby węzłów w drzewie stanów. Dla instancji o rozmiarach $n \geq 24$ skala problemu wywołała gwałtowną eksplozję kombinatoryczną, która mimo zastosowania optymalizacji uniemożliwiła wyznaczenie wyniku optymalnego na standardowej

stacji roboczej. Choć wprowadzenie ograniczenia górnego (UB) pozwoliło znacząco przesunąć granice wykonalności, co widać na przykładzie sukcesu przy instancji gr17, przy większych danych złożoność silniowa okazała się barierą nie do przejścia.

3. Wnioski końcowe

Analiza wydajności algorytmu jednoznacznie wskazuje na dominującą rolę strategii Best-First Search, która w połączeniu z ograniczeniem górnym (UB) pozwoliła na skuteczne rozwiązanie nawet rzeczywistych instancji TSPLIB (gr17, gr21, gr24). Szczególnie istotnym wnioskiem jest obserwacja, jak asymetria kosztów (ATSP) ułatwia proces optymalizacji w porównaniu do wariantu symetrycznego (TSP). W instancjach asymetrycznych zróżnicowane wagi krawędzi naturalnie tworzą duże różnice, które pozwalają algorytmowi na generowanie bardzo precyzyjnych ograniczeń dolnych i szybką eliminację nieatrakcyjnych ścieżek. W przeciwieństwie do nich, warianty symetryczne cechują się dużą redundancją i zbliżonymi kosztami wielu gałęzi, co sprawia, że drzewo stanów jest trudniejsze do „docięcia”, a algorytm musi eksplorować znacznie szerszy obszar poszukiwań przed znalezieniem optimum.

Jednocześnie testy ujawniły fizyczną barierę wydajnościową przy instancjach o rozmiarze $n > 24$, gdzie silniowy przyrost liczby stanów doprowadził do nasycenia pamięci operacyjnej. Zidentyfikowano, że głównym wąskim gardłem nie była moc procesora, lecz kosztowne zarządzanie pamięcią RAM przy głębokim kopiowaniu struktur danych, co w warunkach wielozadaniowości systemu Windows 10 uniemożliwiło stabilną pracę przy największych plikach. Stanowi to cenny wniosek inżynierski: choć zaimplementowany algorytm świetnie radzi sobie z danymi o średniej skali, przejście do profesjonalnych instancji logistycznych wymagałoby wdrożenia jeszcze bardziej rygorystycznego zarządzania zasobami oraz struktur danych minimalizujących narzut na kopiowanie macierzy redukcji.

Literatura

- [1] Applegate D. L. i in., *The Traveling Salesman Problem: A Computational Study*, Princeton University Press, 2006.
- [2] Łyczek Mateusz, *Metoda podziału i ograniczeń*, 2011.
- [3] Clausen Jens, *Branch and Bound Algorithms - Principles and Examples*, 1999.
- [4] Morrison D. R. i in., *Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning*, 2016.
- [5] Sysło M. M. i in., *Algorytmy optymalizacji dyskretnej*, PWN, 2001.